SITEPROOF

Technical Specification — Services, Mobile Screens & Automation

---

Build-Level Detail for the Prototype Team

KeepCodeIn · Version 1.1 · August 2026

Muhammad Khurram — Mobile · Sufyan Baig — AI Services · Asad Ullah — Automation

Contents

0. How to Read This Document

This is the build-level companion to the product doc. It answers three questions the team asked: what backend services do we need, what screens does the mobile app have, and what exactly does the automation do. Each section is written so the owning engineer can start building from it.

- **Section 1:** The AI & backend services (Sufyan) — every service, its job, and its key API endpoints.
- **Section 2:** The mobile screens (Khurram) — every screen, its elements, actions, and states, with mockups.
- **Section 3:** The admin / monitoring web app (Sufyan) — where jobs, pass/fail, and issues are watched, with a mockup.
- **Section 4:** The automation workflows (Asad) — every n8n workflow, its trigger, steps, and output.
- **Section 5:** One end-to-end walkthrough, with a flow diagram, that stitches it all together.
- **Section 6:** A simple start-to-end example in plain language — what the start is and what the end is.

Scope note: only items marked MVP are required for the prototype. Everything else is labelled Later and should be ignored until the demo works.

1. Backend & AI Services

The platform is a set of small services behind one REST API. For the prototype they can live in a single codebase (a modular monolith) and be split later. All services share one PostgreSQL database (with pgvector) and one object store.

1.1 Service map

Service	Responsibility	MVP?
Auth & Identity	Login, tokens, roles (worker / supervisor / owner), org membership	MVP
Org & Settings	Organisation record, branding, plan, vertical selection	MVP (minimal)
Rulebook Ingestion	Upload SOP/code PDFs, chunk, embed, index into vector store	MVP
Media & Storage	Receive photos + audio, store in object store, return refs	MVP
Extraction	Vision on photos + speech-to-text on audio → structured findings	MVP
Compliance (RAG)	Retrieve rulebook clauses, produce verdict + citation + severity	MVP
Job	Create/close jobs, attach findings, hold job state	MVP
Report	Assemble job + findings into a report record (PDF via automation)	MVP
Event / Webhook	Emit job.closed and other events to the automation engine	MVP
Query / Dashboard	Web dashboard data + chat over rulebooks and past jobs	MVP (basic)
Analytics	Compliance trends, team stats, usage metrics	Later
Billing	Plans, usage limits, invoicing	Later

1.2 Key services in detail

1.2.1 Auth & Identity

- **Job:** Authenticate users and gate every API call by role. A worker can capture and close jobs; a supervisor/owner can also upload rulebooks and view all jobs.
- **Notes:** Use a drop-in provider (Clerk / Supabase Auth) to save time. Issue JWTs the mobile app stores securely.

Endpoints:

Endpoint	Purpose
`POST /auth/login`	Email + password → JWT
`POST /auth/refresh`	Refresh an expired token
`GET /me`	Current user, role, org
`POST /orgs/:id/invite`	Supervisor invites a worker (Later: MVP can pre-seed users)

1.2.2 Rulebook Ingestion

- **Job:** Turn a company's SOP/code documents into a searchable knowledge base the Compliance service can query.
- **Pipeline:** Upload PDF → extract text → chunk (by section/clause) → embed each chunk → store vector + source text + clause reference in pgvector.
- **Why it matters:** This is the whole differentiator — the AI reasons against THIS company's rulebook, not a generic template. Keep clause references clean so verdicts can cite them.

Endpoints:

Endpoint	Purpose
`POST /rulebooks`	Create a rulebook (title, vertical)
`POST /rulebooks/:id/documents`	Upload a source PDF; triggers chunk + embed
`GET /rulebooks/:id/status`	Ingestion progress
`GET /rulebooks/:id/search?q=`	Debug: see which clauses a query retrieves

1.2.3 Extraction

- **Job:** Convert raw capture (photos + voice note) into a structured finding the Compliance service can reason about.
- **Steps:** Speech-to-text on the audio (Whisper); vision model on each photo with a prompt that returns structured attributes (object, condition, location, apparent issue); merge into one findings JSON.
- **MVP simplification:** Show the extracted attributes to the worker for a quick confirm before running the verdict — this keeps trust high while the models are unproven.

Endpoints:

Endpoint	Purpose
`POST /extract`	media_refs + audio_ref → structured findings JSON

1.2.4 Compliance (RAG) — the core

- **Job:** Given a structured finding, decide pass / review / fail against the rulebook and cite the exact clause.
- **Steps:** Embed the finding → vector-search the rulebook for the most relevant clauses → pass finding + retrieved clauses to the LLM → return a verdict, the cited clause text, a severity, and a short reason.
- **Output contract:** verdict (pass|review|fail), severity (low|med|high), cited_clause (text + ref), reason (one line). Always return the cited text so the worker and the report can show it.

Endpoints:

Endpoint	Purpose
`POST /compliance/check`	finding + rulebook_id → verdict object
`POST /jobs/:id/findings`	Store a finding + its verdict against a job

1.2.5 Job, Report & Events

- **Job service:** Owns the lifecycle: open a job, attach findings, close a job. Closing a job is the moment everything downstream fires.
- **Report service:** On close, assembles the job + all findings + verdicts into a report record; the actual PDF is rendered by the automation engine so the branding lives in one place.
- **Event service:** Emits job.closed (and later finding.failed) as a webhook the automation engine subscribes to. Payload = job id, org, findings with verdicts, cited clauses.

Endpoints:

Endpoint	Purpose
`POST /jobs`	Start a job (site, assigned worker)
`POST /jobs/:id/close`	Close job → creates report record → emits job.closed webhook
`GET /jobs/:id`	Full job with findings + verdicts (dashboard + report)
`GET /jobs?status=`	List jobs for dashboard

1.2.6 Query / Dashboard

- **Job:** Powers the web dashboard: list jobs, view a report, and a chat box to ask questions over rulebooks and past jobs ("show every job that failed on wiring this month").
- **MVP:** Job list + single report view + basic rulebook chat. Analytics come later.

2. Mobile App Screens

React Native + Expo. The app is deliberately small — the field worker should be able to run a whole job in a few taps, even with no signal. Below is every screen for the MVP, its elements, the actions on it, and the states it can be in.

2.1 Screen list

#	Screen	Purpose	MVP?
1	Login	Sign in, store token	MVP
2	Job list / Home	See assigned jobs, start a new one, sync status	MVP
3	Start job	Pick site / enter job details, choose rulebook	MVP
4	Capture finding	Camera + voice note for one finding	MVP
5	Confirm finding	Review extracted attributes before checking	MVP
6	Verdict	See pass / review / fail + cited clause	MVP
7	Job summary	All findings, then close job	MVP
8	Sync / offline banner	Queue state, retry (component, not a page)	MVP
9	Profile / settings	User, org, logout	MVP (minimal)

2.2 Screens in detail

Screen 1 — Login

- **Elements:** Logo, email field, password field, sign-in button, error text.
- **Actions:** Sign in → store JWT securely → go to Job list.
- **States:** Idle, loading, error (bad credentials / no network).

Screen 2 — Job list / Home

- **Elements:** List of assigned jobs (site, status, date), a prominent "Start job" button, a sync-status chip in the header.
- **Actions:** Tap a job to resume it; tap Start job → Start job screen; pull to refresh.
- **States:** Has jobs / empty; online / offline (chip shows queued count); syncing.

Screen 3 — Start job

- **Elements:** Site / address field, job type, rulebook selector (pre-filled for the single-vertical MVP), start button.
- **Actions:** Create job (POST /jobs, or queue if offline) → go to Capture.
- **States:** Valid / invalid form; creating; queued-offline.

Screen 4 — Capture finding (the heart of the app)

- **Elements:** Full-screen camera, shutter, thumbnail strip of photos taken, a big press-and-hold voice-record button with live transcript hint, "Check finding" button.
- **Actions:** Take one or more photos; hold to record a voice note; tap Check → upload media (or queue) → call /extract.
- **States:** Camera ready; capturing; recording; uploading; offline (queued, shows a badge).
- **Offline behaviour:** Everything is saved locally first; the finding is queued and syncs when signal returns. Never block the worker.

Screen 5 — Confirm finding

- **Elements:** The photos, the transcript, and the extracted attributes (object, condition, location, apparent issue) shown as editable chips.
- **Actions:** Fix any wrong attribute → "Confirm & check" → calls /compliance/check.
- **States:** Extracting; ready to confirm; checking.

- **Why:** Keeps trust high while vision/voice are unproven; can be auto-skipped later once accuracy is high.

Screen 6 — Verdict

- **Elements:** A big colour-coded result — green pass / amber review / red fail — the cited clause text, the one-line reason, and buttons: "Add another finding" or "Back to job".
- **Actions:** Store finding + verdict against the job (POST /jobs/:id/findings); add another or return to summary.
- **States:** Pass / review / fail; stored; error (retry).

Screen 7 — Job summary & close

- **Elements:** List of all findings with their verdict pills, a count of pass/review/fail, and a "Close job" button.
- **Actions:** Close job (POST /jobs/:id/close) → this fires the automation → confirmation screen.
- **States:** Open with findings; closing; closed (report generating).

Screen 8 — Sync / offline banner (component)

- **Elements:** A slim banner showing queued items and a manual retry.
- **Behaviour:** Auto-syncs on reconnect; idempotent uploads so nothing double-posts.

Screen 9 — Profile / settings

- **Elements:** Name, org, role, app version, logout.

2.3 Screen mockups

Low-fidelity mockups of the eight core screens, in the order a worker moves through them.

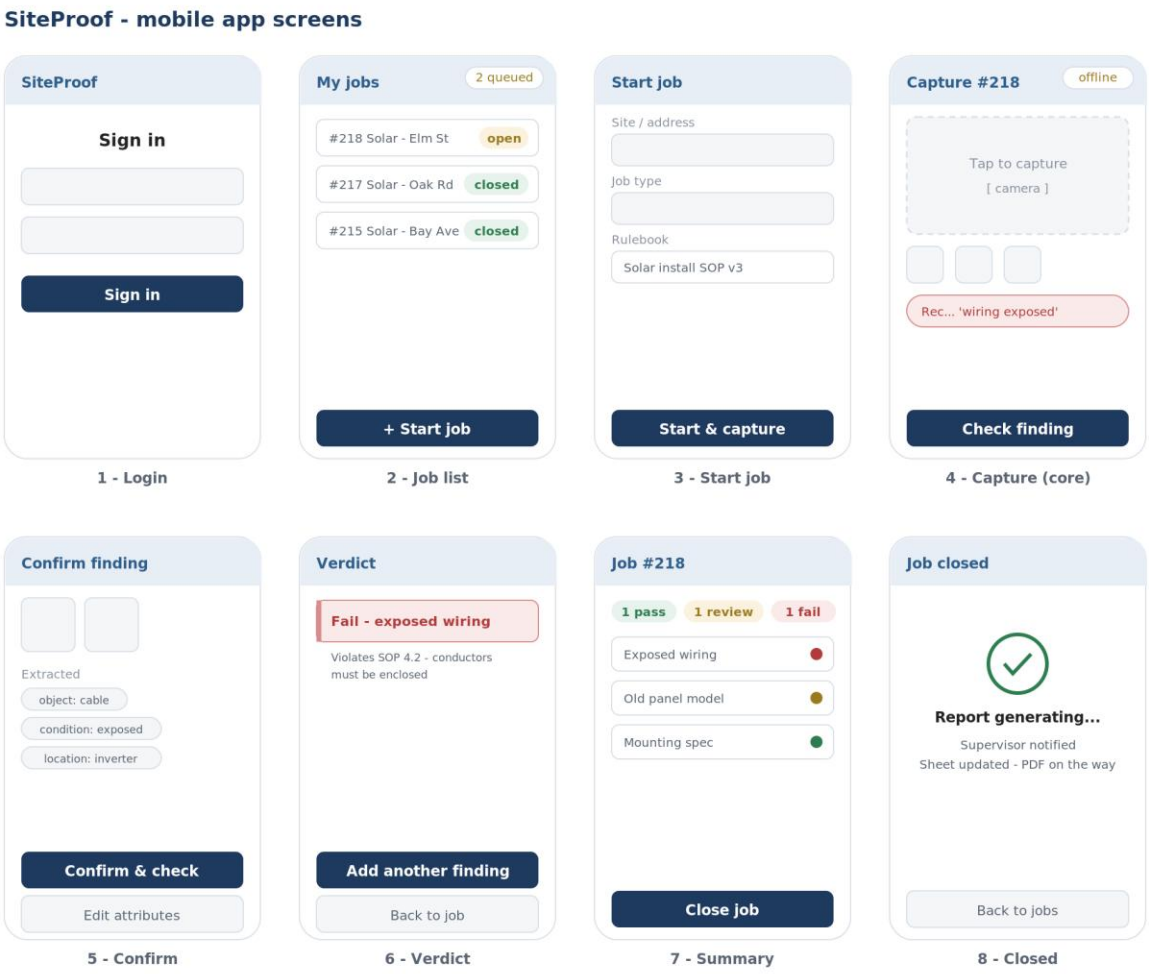

Figure 1 — SiteProof mobile app: the eight MVP screens.

3. Admin / Monitoring Web App

A separate web surface for owners and supervisors to monitor everything from the office. Where the mobile app is for capturing in the field, the admin app is for oversight — who ran what job, what passed or failed, what needs action, and what the automation did. It is part of Sufyan's platform (same API, same database) with a Next.js front end.

3.1 Why it's needed

Field workers close jobs; someone has to watch the whole operation. The admin app answers, at a glance: how many jobs today, how many passed vs failed, which failures still need action, and are the rulebooks up to date. Without it, the pass/fail data is trapped in individual reports.

3.2 Sections

Section	What the admin sees / does	MVP?
Dashboard	Live counts (today, passed, review, failed) + a feed of recent jobs with status	MVP

Section	What the admin sees / does	MVP?
Jobs	Full searchable/filterable table of every job — by status, worker, site, date; open any job	MVP
Job detail	All findings with photo, transcript, verdict, cited clause; the PDF; the automation action log	MVP
Issues queue	Only the review + fail findings that need action; assign and mark resolved	MVP (basic)
Rulebooks	Upload / version SOP & code docs, see ingestion status, test a query	MVP
Team	Workers, their jobs, activity, pass/fail rate	Later
Integrations	Configure routing targets, CRM / Sheet / Slack connections	Later
Analytics	Fail rate by job type / worker / site, compliance trend over time	Later
Settings	Org, branding, roles, alert channels	MVP (minimal)

3.3 Monitoring pass / fail — the core view

- **Dashboard:** Four headline numbers (today, passed, review, failed) so an owner sees the health of the day in one glance, plus a live list of recent jobs colour-coded by status.
- **Jobs table:** Every job as a row: id, site, worker, finding count, a pass/review/fail status pill, and a link to the PDF. Filter by status to see, say, only today's failures.
- **Job detail:** Click a failed job to see exactly which finding failed, the photo, the cited clause, and — importantly — the automation action log confirming the supervisor was notified and the CRM was updated.
- **Issues queue:** A worklist of everything still open (review + fail) so nothing slips; assign to a person and mark resolved when fixed.

3.4 Admin dashboard mockup

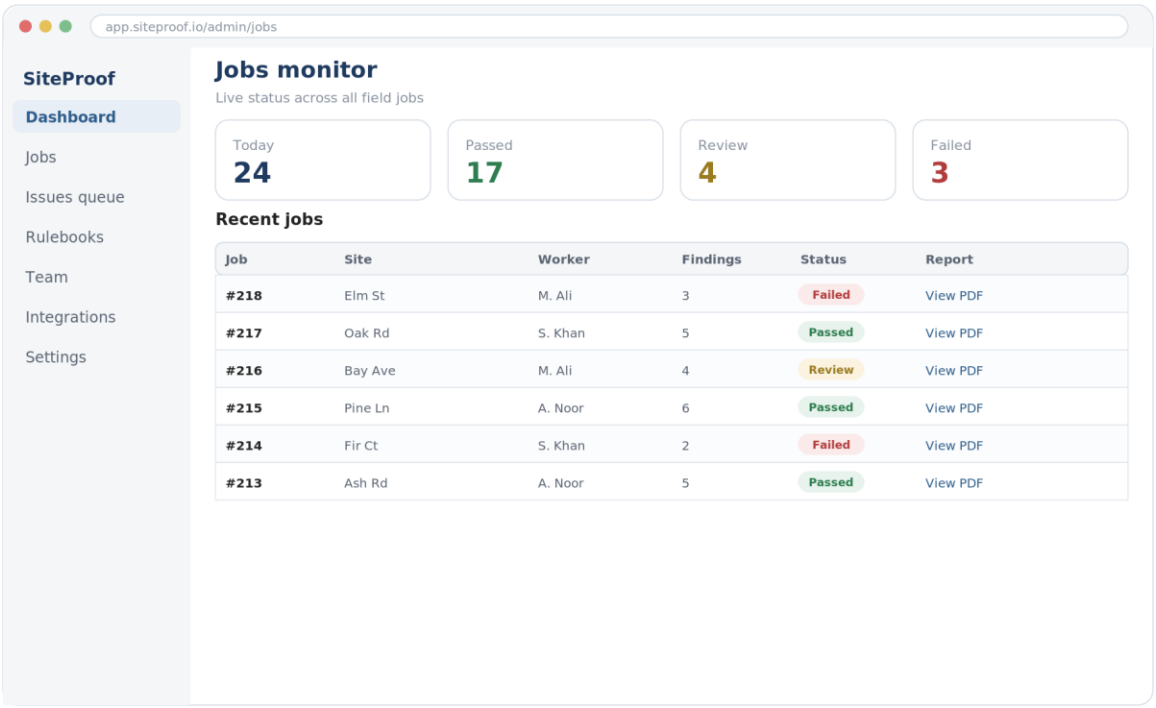

Figure 2 — Admin app: jobs monitor with pass / review / fail at a glance.

Owner: Sufyan (same platform + API). It reuses the Job, Report, and Action data the mobile app and automation already produce — so building it is mostly front end, not new backend.

4. Automation Workflows (n8n)

Everything after "Close job" is automation. One master workflow listens for the job.closed webhook and fans out to sub-workflows. The goal: zero manual typing between the field and the customer's systems.

4.1 Workflow map

Workflow	What it does	MVP?
Job-close listener	Receives job.closed webhook, validates, fans out	MVP
Report generation	Builds a branded PDF from findings + verdicts + cited clauses	MVP
Issue routing	Sends failed/review findings to the right supervisor	MVP
Critical alert	Immediate not: any high-severity fail	MVP (one channel)
System update	Push job outcome to CRM / Google Sheet / calendar	MVP (one target)
Follow-up scheduling	Create a re-inspection task/calendar event for fails	Later
Client delivery	Email the branded report to the end client	Later

4.2 Workflows in detail

4.2.1 Job-close listener (master)

- **Trigger:** Webhook node on job.closed from the Event service.

- **Steps:** Validate payload → fetch full job via GET /jobs/:id if needed → branch into report, routing, alert, and system-update sub-flows.
- **Output:** Kicks off all downstream workflows; writes an Action record per side-effect for the audit trail.

4.2.2 Report generation

- **Trigger:** Called by the master flow.
- **Steps:** Assemble job header + each finding (photo, transcript, verdict, cited clause, reason) into an HTML template → render to PDF → upload to storage → attach PDF ref to the report record.
- **Output:** A branded compliance PDF stored and linked to the job; visible in the dashboard.
- **Nodes:** Function/template node → HTML-to-PDF node → storage upload → HTTP PATCH report record.

4.2.3 Issue routing

- **Trigger:** Called by master flow with the list of findings.
- **Steps:** Filter to review + fail findings → look up the responsible supervisor by job type / severity → send a summary (email or Slack) with the specific findings and a link to the report.
- **Output:** The right person is notified about exactly what needs action — no digging.

4.2.4 Critical alert

- **Trigger:** Any finding with verdict = fail and severity = high.
- **Steps:** Immediate message to a set channel (email/Slack) with the site, the finding, and the cited clause.
- **Output:** Fast escalation on safety-critical issues; separate from the routine routing email.

4.2.5 System update

- **Trigger:** Called by master flow.
- **Steps:** Map the job outcome to the customer's system of record — for MVP, append a row to a Google Sheet or update a CRM record (job id, site, pass/review/fail counts, report link).
- **Output:** The customer's existing tools reflect the job with no manual entry.

4.3 Automation design principles

- Idempotent: re-running on the same job.closed event must not double-send or double-file.
- Observable: every side-effect writes an Action record (type, target, status) so failures are visible.
- Config-driven: routing targets and channels live in config per org, not hard-coded in nodes.
- Fail-soft: if one sub-flow fails (e.g. CRM down), the others still complete and the failure is logged for retry.

5. End-to-End Walkthrough

One finding, all the way through, to show how the three layers hand off. The diagram below maps the whole loop; the numbered steps narrate it.

Figure 3 — End-to-end flow across mobile, services, and automation.

1. Worker opens Job #218, taps Capture, photographs an exposed cable and says "wiring exposed near the inverter." *(Mobile — Khurram)*

2. App uploads the photo + audio (or queues if offline). Extraction returns { object: cable, condition: exposed, location: inverter }. Worker confirms. *(Services — Sufyan)*

3. Compliance embeds the finding, retrieves SOP §4.2, and returns fail / high / "conductors must be enclosed." The app shows a red flag with the clause. *(Services — Sufyan)*

4. Worker adds two more findings, then taps Close job. The Job service emits job.closed. *(Mobile + Services)*

5. n8n catches the webhook: generates the PDF, emails the supervisor the failed finding, fires a critical alert on the high-severity fail, and appends a row to the ops Google Sheet. *(Automation — Asad)*

6. Within a minute the report is in the dashboard and the right people know — with no one typing anything.

Ownership recap

- **Sufyan (Services + Admin app):** Auth, Rulebook Ingestion, Extraction, Compliance/RAG, Job/Report/Event, and the admin monitoring web app.
- **Khurram (Mobile):** Login, Job list, Start job, Capture, Confirm, Verdict, Job summary, offline sync.
- **Asad (Automation):** Job-close listener, Report PDF, Issue routing, Critical alert, System update.
- **Shared:** The API contract, the job.closed webhook payload, and the weekly integration checkpoint.

6. A Simple Example — Start to End

The plainest possible telling of one real job, so anyone can follow what SiteProof does. The strip below shows the whole thing; the text underneath spells out the start and the end.

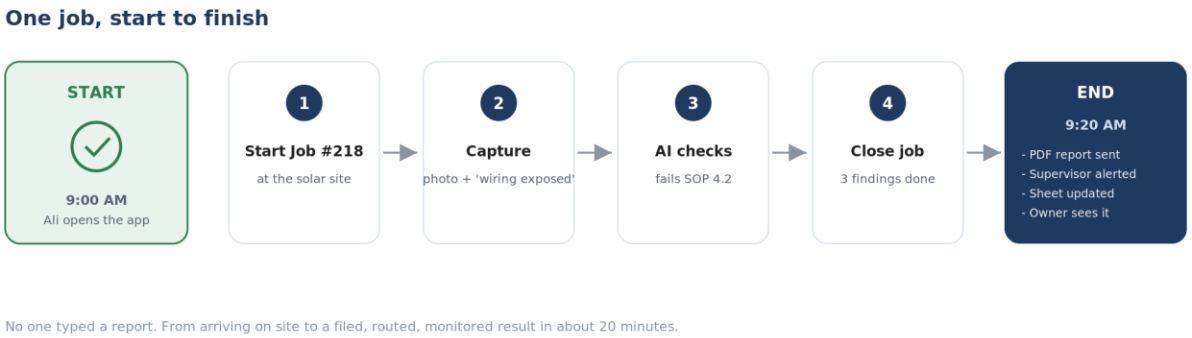

Figure 4 — One solar-install inspection, from arriving on site to a filed result.

The START

9:00 AM. Ali, a field worker, arrives at a house for a solar-panel install inspection. He opens the SiteProof app and taps Start job for Job #218. That is the beginning — a worker on site with a phone. Nothing else is set up in advance.

The middle (a few taps)

- He photographs an exposed cable near the inverter and says out loud, "wiring exposed near the inverter."
- The app reads the photo and voice, and checks it against the company's solar rulebook.
- It instantly shows a red FAIL — "violates SOP §4.2, conductors must be enclosed."
- Ali adds two more findings (one passes, one to review), then taps Close job.

The END

9:20 AM. The moment Ali closes the job, everything happens on its own: a branded PDF report is generated, the supervisor is emailed the failed finding, the office Google Sheet updates to show Job #218 = Failed, and the owner sees it live on the admin dashboard. That is the end — a filed, routed, monitored result with the right people informed.

The point: the worker only captured and closed. No one wrote a report, no one re-typed anything into another system, and nothing was missed. Start = a worker on site. End = a complete compliance record everyone can see — in about 20 minutes.